Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

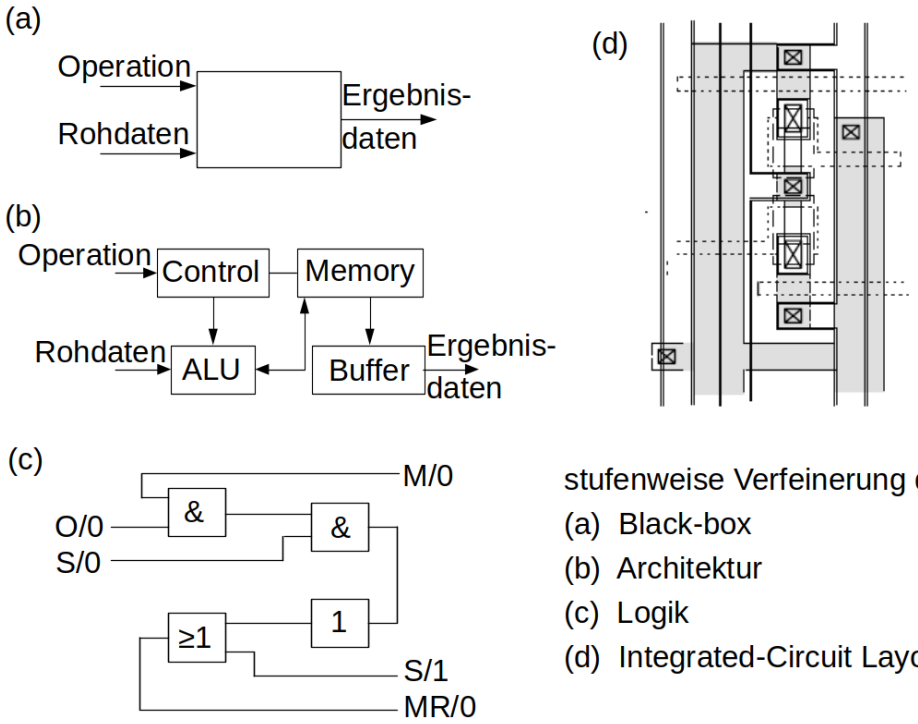
Hardware-Software-Systeme

Kapitel 02 - Entwurf integrierter Schaltungen mit Hardware-Beschreibungssprachen

Prof. Dr.-Ing. Guillermo Payá Vayá, 7. November 2023

Hierarchischer Entwurfsablauf

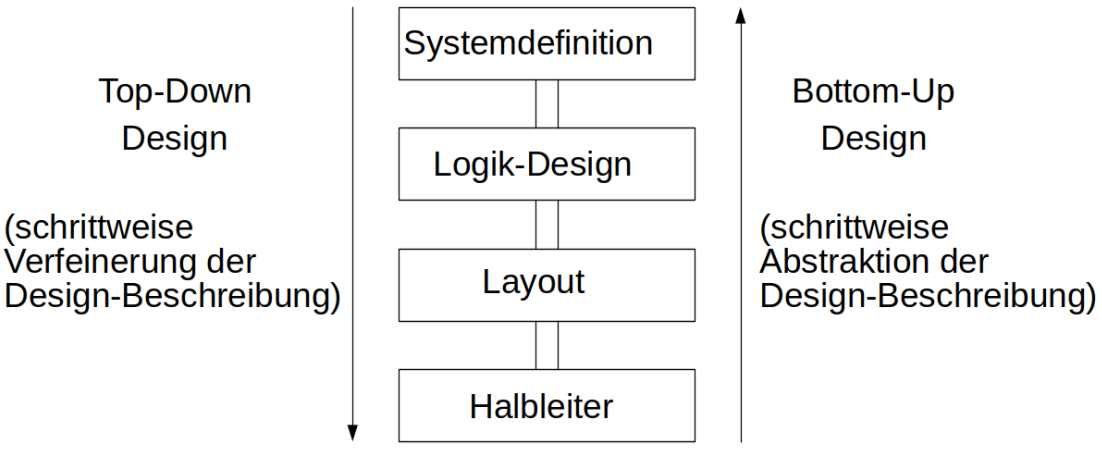
Hierarchischer Entwurfsstil



stufenweise Verfeinerung des Entwurfs:

(a) Black-box
(b) Architektur
(c) Logik
(d) Integrated-Circuit Layout

Entwurfsstrategien



[1]

Hardware-Beschreibungssprachen

Hauptfunktion einer HDL

Hardware Description Language (HDL)

Eine *Hardware Description Language* ist eine formale Sprache, mit der logische Funktionen für ihre Abbildung in eine bestimmte Hardware, z.B. FPGA oder ASIC, beschrieben werden können.

- Die wichtigsten HDLs sind
  - VHDL (Very High Speed Integrated Circuit HDL)
  - Verilog
  - Chisel
- Das Verhalten der Hardware kann auf unterschiedlichen Abstraktionsebenen beschrieben werden
  - *Funktionale Beschreibung* - auf einer hohen Ebene mit wenig Detailinformationen und einer hohen Abstraktion von Hardware
  - *Strukturelle Beschreibung* - auf einer geringen Abstraktionsebene mit mehr Detailinformationen

Geschichte von VHDL (I)

1980 Das US-Verteidigungsministerium verwendet eine Standard HDL (VHDL) um Schaltungsdesign selbst zu dokumentieren, einen gemeinsamen Design-Flow zu verfolgen und bei neuen Technologien alte Schaltungsentwürfe wieder verwenden zu können.

1983 IBM, Texas Instruments, Intermetrics und das US-Verteidigungsministerium schließen sich zusammen, um VHDL und entsprechende Simulations-Tools zu entwickeln.

1987 Das US-Verteidigungsministerium fordert, dass alle digitalen Schaltungen, die für das Verteidigungsministerium neu entwickelt werden, in VHDL beschrieben werden müssen. Der IEEE verabschiedete VHDL als Standard 1076.

Geschichte von VHDL (II)

1993 Der Standard VHDL wird durch IEEE 1164 zu IEEE 1076-1993 (VHDL'93) erweitert. Diese Version ist die am weitesten verbreitete.

1996 Kommerzielle Synthese- und Simulations-Tools sind verfügbar. Weitere Pakete zur Synthese werden dem Standard hinzugefügt.

1999 Erweiterung für analoge und gemischte Schaltungen (VHDL-AMS).

2002 Einführung geschützter Typen und Änderungen für mehr Flexibilität (VHDL-2002: aktueller Standard bei Mainstream-Simulationswerkzeugen).

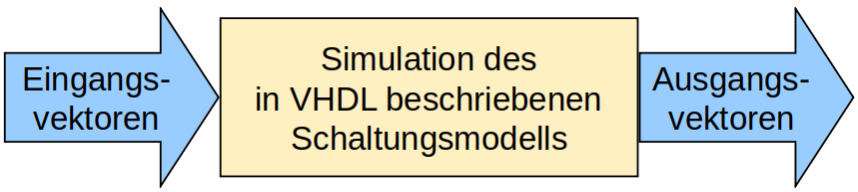
2006 Mehr Operatoren, größere Flexibilität, Integration von Schnittstellen zu C/C++. Alles für eine effizientere Nutzung auf Systemebene (VHDL-2006, Draft 3.0).

Geschichte von VHDL (III)

2009 Veröffentlichung von VHDL 4.0 (VHDL-2008): Ressourcen für IP-Schutz, neue Typen für Fest- und Gleitkommazahlen, Korrekturen und Verbesserung bzgl. der Neuerungen aus VHDL 3.0. Integriert alle bereits vorhandenen sog. std-Packages. Wird einige Jahre dauern, bis er von den Tool-Herstellern komplett unterstützt wird.

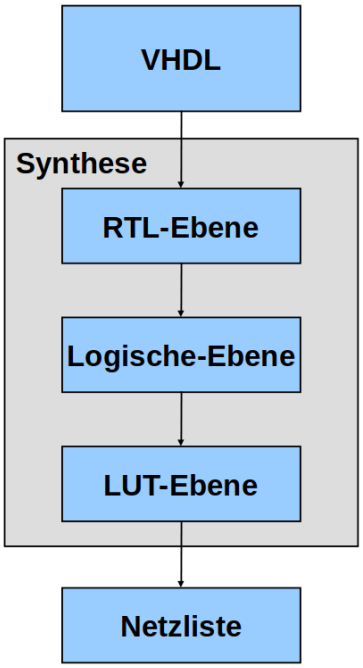
Electronic Design Automation Tools: *Simulation*

- Simulatoren dienen der funktionalen Verifikation einer Schaltungsbeschreibung auf verschiedenen Abstraktionsebenen
- Es kann sowohl das logische als auch das zeitliche Verhalten simuliert werden
- Zur Simulation des zeitlichen Verhaltens müssen:
  - entweder vor der Synthese Zeitinformationen im VHDL-Programm integriert werden
  - oder nach der Synthese werden diese Informationen aus den Zellbibliotheken der FPGA- bzw. ASIC-Hersteller bezogen
- Zeitliche Simulationen sollten nur nach der Synthese durchgeführt werden



Electronic Design Automation Tools: *Synthesis*

- Die Synthese ist das Überführen einer VHDL-Beschreibung über verschiedene Zwischenstufen in eine Netzliste, die die FPGA-Konfiguration beschreibt. Auf jeder Zwischenstufe werden logische Optimierungstechniken angewendet.
- Einige Sprachkonstrukte, die in der Simulation sinnvoll sein können (z.B. zeitliches Verhalten, Fließkommazahlen oder Operationen zur Bearbeitung von Dateien) werden von den Synthese-Tools nicht unterstützt



Andere Arten von HDL

Hardware Verification Languages

Hardware Verfication Languages (HVL) [2]

Eine *Hardware Verification Language* ist eine Programmiersprache, die zur Verifikation der Entwürfe elektronischer Schaltungen verwendet wird, die in einer Hardware-Beschreibungssprache beschrieben wurden. HVLs enthalten typischerweise Merkmale einer High-Level-Programmiersprache wie C++ oder Java sowie Funktionen zur einfachen Manipulation auf Bitebene, die denen von HDLs ähneln.

- Die wichtigsten *Features* sind
  - Random generation
  - Temporal assertions
  - Functional coverage constructs
- Die wichtigsten HVLs sind
  - System-Verilog
  - System-C
  - Vera/OpenVera

Architecture Description Languages

Architecture Description Language (ADL) [3]

Eine ADL enthält in der Regel Sprachelemente, die erforderlich sind, um sowohl die *Instruktionssatzarchitektur* eines Prozessors (z. B. Instruktionscodierung, Syntax, Semantik und Latenzen usw.) als auch seine *Mikroarchitektur* (z.B. die Speicher- und Registerstruktur, die Pipelinestruktur, das Verhalten der Instruktionen usw.) zu beschreiben, aus denen sowohl die RTL-Hardware als auch das Software-Ökosystem automatisch abgeleitet werden können.

- Beispiele von ADL sind
  - LISA 2.0
  - nML
  - EXPRESSION

Analog und Mixed-Signal (AMS) HDLs

- **AMS-HDL**-Standards wurden entwickelt, um Entwicklern von analogen und Mixed-Signal-Systemen sowie von integrierten Schaltungen die Erstellung und Verwendung von Modulen zu ermöglichen, die sowohl High-Level-Verhaltensbeschreibungen als auch strukturelle Beschreibungen von Systemen und Komponenten kapseln.
- AMS-HDLs bieten sowohl eine zeitkontinuierliche als auch eine Event-gesteuerte Modellierungssemantik und ist daher für analoge, digitale und gemischte analog/digitale Schaltungen geeignet.
- Sie eignen sich besonders gut für die Verifizierung sehr komplexer analoger, Mixed-Signal- und Hochfrequenz-Schaltungen.
- Die wichtigsten AMS-HDLs sind
  - VHDL-AMS
  - Verilog-AMS

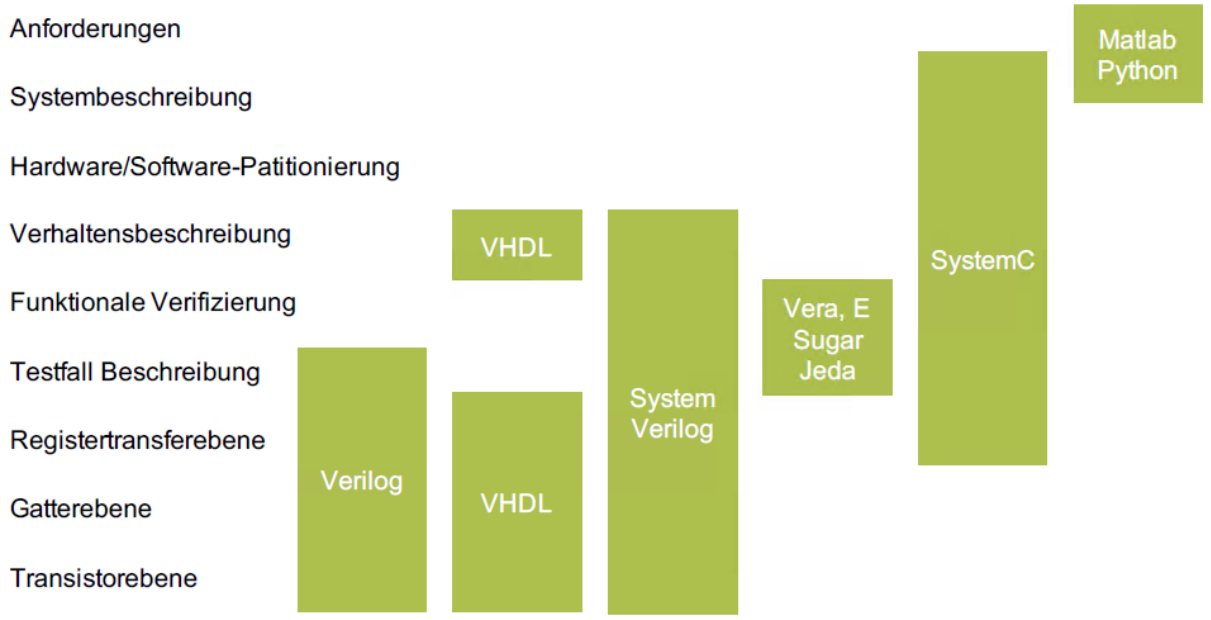
High-Level Synthesis

High-Level Synthesis (HLS) [4]

High-Level-Synthese, auch bekannt als Verhaltenssynthese und algorithmische Synthese, ist ein Entwurfsprozess, bei dem eine funktionale Beschreibung eines Entwurfs auf High-Level-Ebene automatisch in eine RTL-Implementierung kompiliert wird, die bestimmte benutzerdefinierte Design-Constraints erfüllt.

- Bei HLS wird (versucht), Parallelität aus der funktionalen Beschreibung zu extrahieren und eine Mikroarchitektur zu konstruieren, die schneller und billiger ist als die einfache Ausführung der funktionalen Beschreibung als Programm auf einem Mikroprozessor.
- Die wichtigsten Sprachen, die von HLS-Tools verwendet werden, sind
  - System-Verilog und System-C
  - C/C++

Vergleich verschiedener HDLs



Literatur

[1] H. Blume, “Vorlesungsskript: FPGA-Entwurfstechnik,” *Leibniz Universität Hannover*, 2020.

[2] J. Bergeron, *Writing testbenches using SystemVerilog*. Springer Science & Business Media, 2007.

[3] K. Karuri and R. Leupers, *Application analysis tools for ASIP design: application profiling and instruction-set customization*. Springer Science & Business Media, 2011.

[4] P. Coussy and A. Morawiec, *High-level synthesis*. Springer, 2010, vol. 1.